INFORME POST-INCIDENTE: MANTENIMIENTO Y RECUPERACIÓN

> **Materia:** Bases de Datos II (TUP) **Motor:** PostgreSQL 18 **SO:** Windows (PowerShell 5.1)
> **Base Producción:** `foodStore` **Base Restauración:** `foodstore_restore_test` **Fecha:** 2026-10-30 23:28:25

1. Resumen Ejecutivo y Metodología

Se ejecutó satisfactoriamente el simulacro de recuperación ante desastres sobre la base descartable aislada `foodstore_restore_test`, protegiendo la base `foodStore`. El procedimiento automatizó la creación de la base receptora, la restauración del catálogo y datos mediante `pg_restore` (con banderas de seguridad `--no-owner --no-privileges`), y la posterior validación de integridad referencial y restricciones lógicas. Las credenciales se gestionaron de forma segura sin exposición en código ni commits (vía `pgpass.conf` y memoria volátil).

2. Cronograma y Métricas de Recuperación (RPO y RTO)

Métrica	Definición Conceptual	Meta / Previsto	Valor Observado	Estado
RPO	Antigüedad máxima admisible de los datos ante un siniestro.	< 24 horas	**06h 00m 26s**	**CUMPLIDO**
RTO	Tiempo total de reconstrucción, restauración y validación.	< 300 s (5 min)	**6.18 segundos**	**CUMPLIDO**

Respaldo: `foodStore_2026-10-30_172759.dump` (Formato custom `-Fc`, 32.62 KB). Cronómetro: 23:28:25 a 23:28:31.

3. Validación de Integridad de Datos y Restricciones Lógicas

Entidad / Regla de Integridad	Filas Producción	Filas Restauradas	Inconsistencias	Resultado
`categoria`	4	4	0	**VÁLIDO**
`usuario` (compradores)	5	5	0	**VÁLIDO**
`producto`	12	12	0	**VÁLIDO**
`pedido`	11	11	0	**VÁLIDO**
`detalle_pedido`	20	20	0	**VÁLIDO**
Control Lógico: `producto` (precio < 0 OR stock < 0)	0	0	0	**VÁLIDO**
Control Lógico: `detalle_pedido` (cant ≤ 0 OR precio < 0)	0	0	0	**VÁLIDO**

4. Monitoreo y Política Técnica de Mantenimiento

Rechazo a `VACUUM FULL`: Prohibido para mantenimiento de rutina por exigir bloqueo de tabla total (`AccessExclusiveLock`), suspendiendo lecturas y escrituras, lo que produce caída del servicio (downtime). Adicionalmente duplica el espacio requerido en disco y satura el I/O. Se aplicó **`VACUUM (VERBOSE, ANALYZE)`** estándar, saneando tuplas muertas a cero (`n_dead_tup = 0`) y actualizando estadísticas del optimizador de forma 100% concurrente.

5. Problemas Reales Encontrados y Soluciones

- **Espacios en rutas Windows:** `pg_dump` fragmentaba argumentos por espacios en el nombre de usuario. Se implementó delimitación y entrecomillado dinámico de argumentos en PowerShell.
- **Sensibilidad a Mayúsculas (`foodStore`):** `pg_dump -d foodstore` fallaba al buscar la base en minúsculas. Se formalizó el parámetro respetando el identificador exacto.
- **Catálogo (`cliente` vs. `usuario`):** La plantilla del TP referenciaba `cliente`, inexistente en el esquema físico. Se alinearon las rutinas a las entidades reales `usuario` y `usuario_anon`.

6. Mejoras Operativas y Verificación del Trabajo Asistido

Mejora propuesta: Implementar archivado continuo de bitácoras WAL (Write-Ahead Logging) para permitir Point-In-Time Recovery (PITR) y reducir el RPO de 24 horas a minutos.
Verificación Humana de Asistencia con IA: Se emplearon herramientas generativas para bosquejar scripts. Cada componente fue revisado y adaptado humanamente: se resolvieron incompatibilidades con el parser de PowerShell 5.1, se impidió la exposición de claves y se probó cada acción en laboratorio antes de darla por válida.